0153. Find Minimum in Rotated Sorted Array

egory: Binary Search | LeetCode: https://leetcode.com/problems/find-minimum-i

Problem

- A sorted array of distinct values was rotated. Return its minimum in O(log n).
- Example: [3,4,5,1,2] → 1.

Intuition

- Compare nums[mid] with nums[hi]. If nums[mid] > nums[hi], the drop (and the minimum) lies to the right of mid.
- Otherwise mid..hi is sorted, so the minimum is at mid or to its left.

Brute Force

- Linear scan for the min: O(n).

Optimal Approach

- Lo = 0, hi = n-1.
- While lo < hi: mid; if nums[mid] > nums[hi], lo = mid+1; else hi = mid.
- Return nums[lo].

Worked Example

```
[4,5,6,7,0,1,2].

lo=0 hi=6 mid=3 (7) > 2 -> lo=4
lo=4 hi=6 mid=5 (1) <= 2 -> hi=5
lo=4 hi=5 mid=4 (0) <= 1 -> hi=4
answer nums[4] = 0
```

Java Solution

```java
public class FindMinimumInRotatedSortedArray {
    public int findMin(int[] nums) {
        int lo = 0, hi = nums.length - 1;
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (nums[mid] > nums[hi]) lo = mid + 1;
            else hi = mid;
        }
        return nums[lo];
    }
}
```

Complexity

- Time: O(log n). Space: O(1).

Pitfalls

- Comparing with nums[lo] instead of nums[hi] fails on an unrotated array unless handled specially.
- Hi = mid (not mid-1): mid could be the minimum.